Personalized Discovery Engine

Project - Netflix Prize DS Recommendation system

Name: Donka Harish Kumar
Enroll : 24125014

The Problem Overview

The Scale and Sparsity Paradox

The Dataset: Worked with the Netflix Prize Dataset (100M+ ratings).
Extreme Sparsity: Calculated at **99.8721%**—users have rated <0.2% of the catalog.
The Long-Tail Bias: 10% of movies get 76% of ratings; 50% of movies get only 2%.
Goal: Build a system that recommends niche discoveries, not just mainstream blockbusters.

Data Engineering

Solving the "Cascading Matrix Collapse"

The Problem: Standard random sampling causes niche movies to vanish from the data.

The Solution: Log-Stratified Inverse-Popularity Sampling. (Binnin method_)

1. Applied a quality filter (>= 5 user ratings, >= 10 movie ratings).
2. Mapped movies into 4 Log-Scale tiers.
3. Oversampled "Long-Tail" titles and downsampled "Blockbusters."

Result: Agile **1.96M row matrix** preserving **100% of the movie catalog**.

Model Development

Headline: Comparing Collaborative Filtering

Architectures

Model A: Item-Based CF (KNN)

- Calculates similarity between movies (4,711 x 4,711 matrix).
- ***Justification****:* Chosen over User-Based CF to prevent RAM exhaustion (240k x 240k).

Model B: Matrix Factorization (SVD)

- Decomposes interactions into latent factors (hidden features).
- *Justification:* Compresses data into dense vectors for faster inference.

Experimental Results

Metric	Item-Based (KNN)	Matrix Factorization (SVD)
Validation RMSE	0.9833	**0.9850**
MAP@10 (Ranking)	**0.6161**	0.6100
Training Time	**2.83s**	13.53s
Prediction Latency	6.94s	**1.88s**

Decision : SVD was selected for deployment.
Reasoning : Sacrificed 0.0017 in RMSE to achieve a **3.7x speedup** in user-facing latency.

Key Insights

- **User Bias Detection:** SVD autonomously identified "harsh critics" vs. "generous raters," scaling recommendations to match individual personality.
- **Latent Clustering:** Without genre tags, the model grouped similar content (e.g., 90s sitcoms) strictly through interaction proximity.
- **Simplicity Wins:** Standard SVD provided the best balance of performance and maintainability, avoiding the overhead of complex temporal models.

Recommendation Examples

- **User 305344 (Prestige Drama):** *24: Season 1*, *Carnivale*, *The Royal Tenenbaums*.
- **User 387418 (90s Pop Culture):** *Friends*, *Coupling*, *Buffy the Vampire Slayer*.
- **Proof of Discovery:** Successfully filtered out seen items and ranked unseen "Discoveries" by projected score.

Conclusion & Future Improvements

- **Headline:** Final Takeaways
- **Summary:** Delivered a mathematically sound, low-latency engine capable of handling extreme sparsity.
- **Future Work:**
  1. **Temporal Bias:** Implement *TimeSVD++* to track how user tastes shift over years (For the models I have used, dont require the time feature…)
  2. **Cold Start:** Integrate a content-based hybrid filter for brand-new movies.
  3. **Deployment:** Convert to a Streamlit Dashboard or API service, Using Proper Back-end services and tools to integrate into a power driven system.